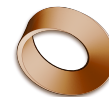


Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 4 — Operator Activities — ANSWERS

Work down the page. You are not expected to reach the bottom.

Warm up

1 • Predict, then run adapted from Paul Long

Write down your answer *before* you touch a keyboard.

Expression	Your prediction	What it gave
80 DIV 11		7
80 MOD 11		3
11 DIV 80		0
11 MOD 80		11

Rows 3 and 4 are the interesting ones. Swapping the numbers round does not swap the answers round.

Before you go further — loops and decisions

Most of what follows needs a loop. Here is the review, because it is easy to mix these two up.

The difference is how many times the question gets asked.

if — asks ONCE

```
if age >= 17:
    print("Old enough")
else:
    print("Not old enough")
```

The question is asked, one branch runs, and the program moves on. **It never comes back.**

while — asks OVER AND OVER

```
while age < 17:
    print("Too young")
    age = age + 1
```

The question is asked, the block runs, and then **it goes back and asks again**. It only stops when the answer is finally no.

The rest of the family, briefly:

Keyword	What it does
if	Asks a question once. Runs its block if the answer is yes.
elif	“Otherwise, try <i>this</i> question.” Only reached if the ones above it were no.
else	“Otherwise do this.” No question — it is the catch-all, and it is optional.
while	Asks a question, runs its block, then asks the same question again .
for	Repeats a known number of times. Use it when you can count in advance.

Every while loop needs three things. Miss one and it breaks.

1. Something set up BEFORE the loop — the variable the question is about has to already exist, or the very first question cannot be asked.

2. A question at the top — something that is either true or false.

3. Something INSIDE that changes the answer. This is the one people forget. If nothing in the block changes the thing being tested, the answer stays yes for ever and the program never stops. That is an **infinite loop**, and the only way out is to close the tab.

Before you run any while loop, point at the line inside it that will eventually make the question false. If you cannot find that line, do not run it.

for or while? Ask yourself: *do I know how many times before I start?* If yes, for. If you only know when to *stop*, while.

Build

2 • Pulling digits out of a number take this one slowly

MOD 10 gives you the last digit of a number. DIV 10 throws that last digit away. Between them, you can take any number completely apart.

```
number = 4827
print(number % 10)      # 7    -- the last digit
print(number // 10)     # 482  -- everything except the last digit
```

(a) Print the last digit of 4827.

7

(b) Print the *second*-from-last digit.

2

Hint: strip one digit off first, then take the last digit of what is left.

(c) **Print the SUM of all four digits of 4827.**

[the long one]

This one is worth doing properly, because the pattern in it turns up everywhere. Work through the steps in order and do not skip to the code.

Step 1 — do it by hand first. Fill in this table. The trick is that whatever `// 10` leaves behind becomes the number you are holding on the next row.

Number I am holding	% 10 gives me	// 10 leaves
4827	7	482
482	2	48
48	8	4
4	4	0

The middle column is your four digits. Add them up — that is the answer you are aiming for. **(21)**

Step 2 — three questions about your table.

- (i) In each row you did the *same two things*. What were they?
- (ii) **How did you know to stop?** You could see 4827 has four digits — but the program cannot. *What was true about the number at the bottom that was not true before?* **It had become 0.**
- (iii) You were adding the digits up as you went. Where did that total live? It was not in the original number — you had to *start* it somewhere and update it each row.

Step 3 — try it with 57 before you write anything.

57 takes **two** passes to reach 0. 4827 takes four. 123456 takes six. **So you cannot write “repeat four times”** — the program is never told how many digits there are. It only knows the number it is holding right now.

That is exactly why this is a while loop and not a for loop.

Step 4 — fill in the four blanks.

```
number = 4827
total  = 0

while _____ :           # is there anything left to strip?
    digit = _____       # the middle column of your table
    total = _____       # add that digit to the running total
    number = _____      # the right-hand column -- what is left

print(total)
```

```
number = 4827
total  = 0

while number != 0 :
    digit = number % 10
    total = total + digit
    number = number // 10

print(total)                # 21
```

Two of these you have already written — digit is part (a) and number is part (b), just with the word number in place of 4827, because it changes every pass.

The line that looks wrong: `total = total + digit`

As maths that is nonsense — nothing equals itself plus something. **But = does not mean “equals”.** It means *put this into that*. Work out the right-hand side first, then drop the answer into the box on the left.

First pass: total is 0 and digit is 7, so the right-hand side is 7, and 7 goes into total. Second pass: total is 7 and digit is 2, so 9 goes into total.

This is called a **running total**. Counting, totalling, averaging — all the same three lines. You will use it constantly.

The bug to avoid — and it is the self-checkout bug from Programming 2.

`total = 0` must sit **above** the loop, not inside it. Indent it into the loop and it resets to zero on every single pass, so you finish with only the last digit.

You would get 4 instead of 21 — and there is **no error message**. The program runs perfectly and gives you the wrong answer. That is a logic error.

Try it both ways on purpose. Seeing it fail once is worth more than being told.

Step 5 — when it works, test it. Try 57 (should give 12), 9 (should give 9) and 1000 (should give 1). **Then try 0.** What happens, and is that right? **With 0 the question is false immediately, the loop body never runs at all, and it prints 0. Which is correct — and a good demonstration that a while loop can run zero times.**

3 • A proper stopwatch

3661 seconds is 1 hour, 1 minute and 1 second. Write a program that turns any number of seconds into hours, minutes and seconds.

Ask the user for a number of seconds. Print it like 1h 1m 1s.

```
total = int(input("Seconds: "))
hours  = total // 3600
minutes = (total % 3600) // 60
seconds = total % 60
print(str(hours) + "h " + str(minutes) + "m " + str(seconds) + "s")
```

The middle line is the one they get wrong: you must take the remainder *first* (what is left after the whole hours) and only then divide by 60.

Test it with: 60 · 3599 · 3600 · 3661 · 86399.

4 • The change machine

A vending machine works in pence. Ask the user for an amount in pence and print it properly.

287 should print as 2 pounds and 87 pence.

Then make it harder: give the change in coins. How many £1, 50p, 20p, 10p, 5p, 2p and 1p? *Each coin is one DIV and one MOD.*

Mark scheme

Worked solution — the change machine. Tested, CPython 3.9.6.

```
pence = int(input("Amount in pence: "))
print(str(pence // 100) + " pounds and " + str(pence % 100) + " pence")
```

```

left = pence
for name, value in [("1 pound",100), ("50p",50), ("20p",20),
                    ("10p",10), ("5p",5), ("2p",2), ("1p",1)]:
    print(name + ": " + str(left // value))
    left = left % value

```

287p gives 2 pounds and 87 pence; then $2 \times \text{£}1$, $1 \times 50\text{p}$, $1 \times 20\text{p}$, $1 \times 10\text{p}$, $1 \times 5\text{p}$, $1 \times 2\text{p}$, $0 \times 1\text{p}$. ($200+50+20+10+5+2 = 287$.)

The pattern is the point: DIV says *how many* of that coin, MOD leaves *what is still owed*, and that carries into the next coin. The same two-step as pulling digits out of a number.

Most students will write it out seven times without a loop. That is fine and still correct — show the loop afterwards as a tidy-up, not a correction.

5 • FizzBuzz the classic — every programmer has written this

Print the numbers 1 to 100, except:

- if the number divides by **3**, print Fizz instead
- if it divides by **5**, print Buzz instead
- if it divides by **both**, print FizzBuzz

The first fifteen lines should read:

1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz

The trap, and it catches nearly everybody. Most people write the test for 3 first, then the test for 5, then wonder why 15 prints Fizz instead of FizzBuzz.

Check for *both* first. Once a number has printed Fizz the decision is already made, so 15 never reaches the FizzBuzz test. Either test `% 15 == 0` at the top, or combine the two conditions with **AND**.

Mark scheme

Worked solution — FizzBuzz. Tested, CPython 3.9.6.

```
for number in range(1, 101):
    if number % 15 == 0:                # BOTH -- must be tested FIRST
        print("FizzBuzz")
    elif number % 3 == 0:
        print("Fizz")
    elif number % 5 == 0:
        print("Buzz")
    else:
        print(number)
```

First twenty: 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz 16 17 Fizz 19 Buzz.

Why % 15? Anything divisible by both 3 and 5 is divisible by 15, so one test does the work of two. Combining the conditions with **AND** is equally correct.

Marking: the ORDER is the mark. Put the 3 test first and 15 never prints FizzBuzz, because `elif` only runs when everything above it failed. Output that is right except at 15, 30 and 45 is exactly this — and it is a *logic* error, with no message.

Also the first for loop on the sheet, and a fair one: you know it runs 100 times before you start, so for is right and while would be clumsy.

Stretch

6 • Leap years

A year is a leap year if it divides by **4** — *except* that century years are not, *unless* they divide by **400**.

So **2024** is a leap year. **1900** is not. **2000** is.

Write a program that asks for a year and says whether it is a leap year. **Three MODs.**

Test it with: 2024 · 2023 · 1900 · 2000.

Mark scheme

Worked solution — leap years. Tested, CPython 3.9.6.

```
year = int(input("Year: "))

if year % 400 == 0:
    print("Leap year")
elif year % 100 == 0:
    print("Not a leap year")
elif year % 4 == 0:
    print("Leap year")
else:
    print("Not a leap year")
```

Tested: 2024 leap · 2023 not · 1900 not · 2000 leap · 2100 not.

The order runs from MOST specific to least. 2000 divides by 400, by 100 and by 4, so whichever test it meets first decides the answer. Put % 4 at the top and 1900 wrongly comes out as a leap year.

Add 2100 as a test. It is the next century year that catches people.

7 • Squares and cubes adapted from Paul Long

Using **:

- (a) Ask for the length of a side and print the **area of the square**.
- (b) Ask for the length of a side and print the **volume of the cube**.
- (c) Ask for a radius and print the **volume of a sphere**. *You will need $V = \frac{4}{3}\pi r^3$ — and a value for π .*

Mark scheme

Worked solution — squares, cubes and spheres. Tested, CPython 3.9.6.

```
side = float(input("Length of side: "))
print("Area of square: " + str(side ** 2))
print("Volume of cube: " + str(side ** 3))

radius = float(input("Radius: "))
pi = 3.14159
print("Volume of sphere: " + str((4 / 3) * pi * radius ** 3))
```

Side 5: area 25, volume 125. **Radius 3:** 113.1 to 1 d.p.

- 1. **float(), not int().** A side can be 2.5 cm. Casting to int gives a wrong answer for every non-whole input, with no error to warn them.
- 2. **(4 / 3) in brackets,** and it gives 1.333... because / always returns a Real — the opening section of Programming 4 paying off.
- 3. **** binds tighter than *,** so radius ** 3 happens first. They do not need the word “precedence” — but if an answer is wildly out, brackets are the first thing to check.

8 • The same fact from both sides Paper 1 meets Paper 2

You learned **binary shifts** for Paper 1 — shift left to multiply by 2, shift right to divide by 2. You have just learned DIV and MOD for Paper 2.

They are the same thing. Python writes a shift as >> and <<.

Operator	Shift	Same?
22 // 2	22 >> 1	both 11 — yes
22 * 2	22 << 1	both 44 — yes
4827 // 8	4827 >> 3	both 603 — yes

Check all three at the machine. Then think about what MOD is doing:

4827 in binary is 1001011011011. Its last bit is 1, and $4827 \text{ MOD } 2$ is 1.

$n \text{ MOD } 2$ **is the last bit of n .** So “is this number even?” really means “is the last bit a 0?” — and $n \text{ MOD } 16$ is the last *four* bits, which is exactly one hexadecimal digit.

So why does the processor care?

A multiply or a divide is genuine work — the hardware has to compute it. **A shift is almost free:** nothing is calculated, the bits are simply wired one column across. It is the fastest thing a processor can do.

So when you write $n // 2$, a good compiler does not divide. **It shifts.** You wrote the Paper 2 operator; the machine ran the Paper 1 one.

This is not on the specification. It is here because the two halves of your course are describing the same machine, and it is worth seeing that once.

Everything on this page uses DIV, MOD or ^ and nothing else you have not already met. If you are stuck, the question is almost always “how many whole ones fit, and what is left over?”